MoveIt2 Simulation-Reality Linkage

Preface: The Raspberry Pi 5 runs in Docker, so the MoveIt2 experience is limited; it is recommended that Raspberry Pi 5 users run MoveIt2-related cases on a virtual machine. The Orin board runs directly on the host, so Orin board users can run MoveIt2-related cases directly on the board, using the same commands as running on a virtual machine.

1. Content Description

This section explains how to combine the simulated robot arm in RViz with the real robot arm, so as to drive the real machine.

2. Preparations

Preface: Since the real robot arm does not have obstacle avoidance, some poses may collide with obstacles. Therefore, before driving the real machine, make sure there are no obstacles around the robot arm.

2.2. Distributed Communication between the Virtual Machine and the Car

The virtual machine and the car need to be able to communicate with each other. Two steps are required to achieve this:

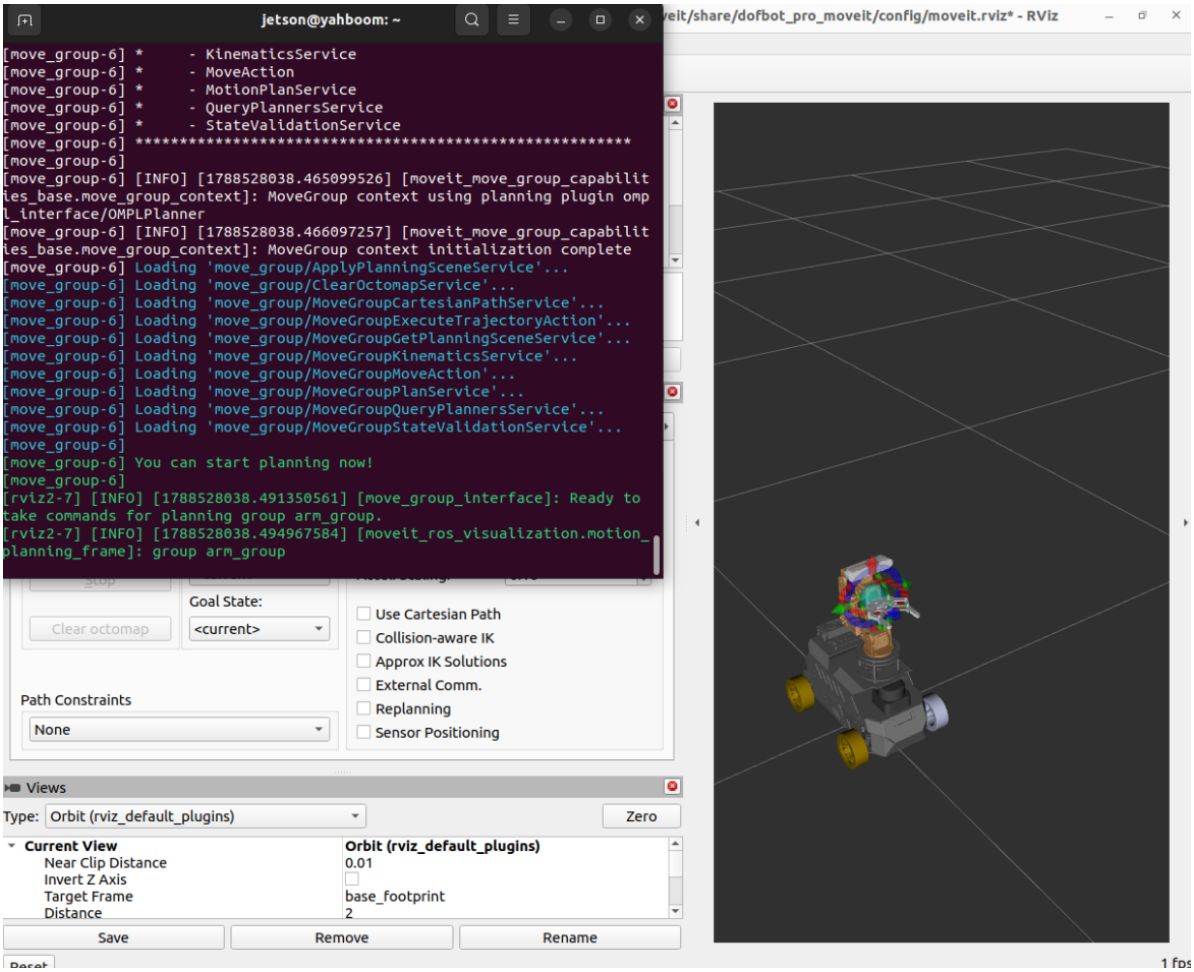
- They must be on the same local area network. The simplest way is to connect to the same Wi-Fi;
- The `ROS_DOMAIN_ID` of both must be consistent. The car's default `ROS_DOMAIN_ID` is 30, and the virtual machine's default `ROS_DOMAIN_ID` is also 30. If the two are different, you need to modify the `ROS_DOMAIN_ID` of the virtual machine. Edit the `~/.bashrc` file, change the value of `ROS_DOMAIN_ID` inside to be the same as the car's, save and exit, then enter the command `source ~/.bashrc` to refresh the environment variables.
- Check whether distributed communication between the two has been established. On the virtual machine, enter `ros2 node list`; if you see **/YB_Node**, it means the two have connected.

3. Launching the Program

Enter the following command in the terminal to launch the MoveIt2 simulation environment:

```
ros2 launch dofbot_pro_moveit demo_3d.launch.py
```

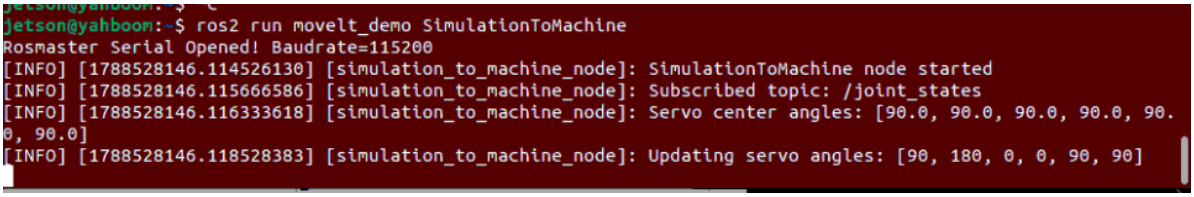
Once the program starts, when **You can start planning now!** appears in the terminal, it means the startup was successful, as shown in the figure below:



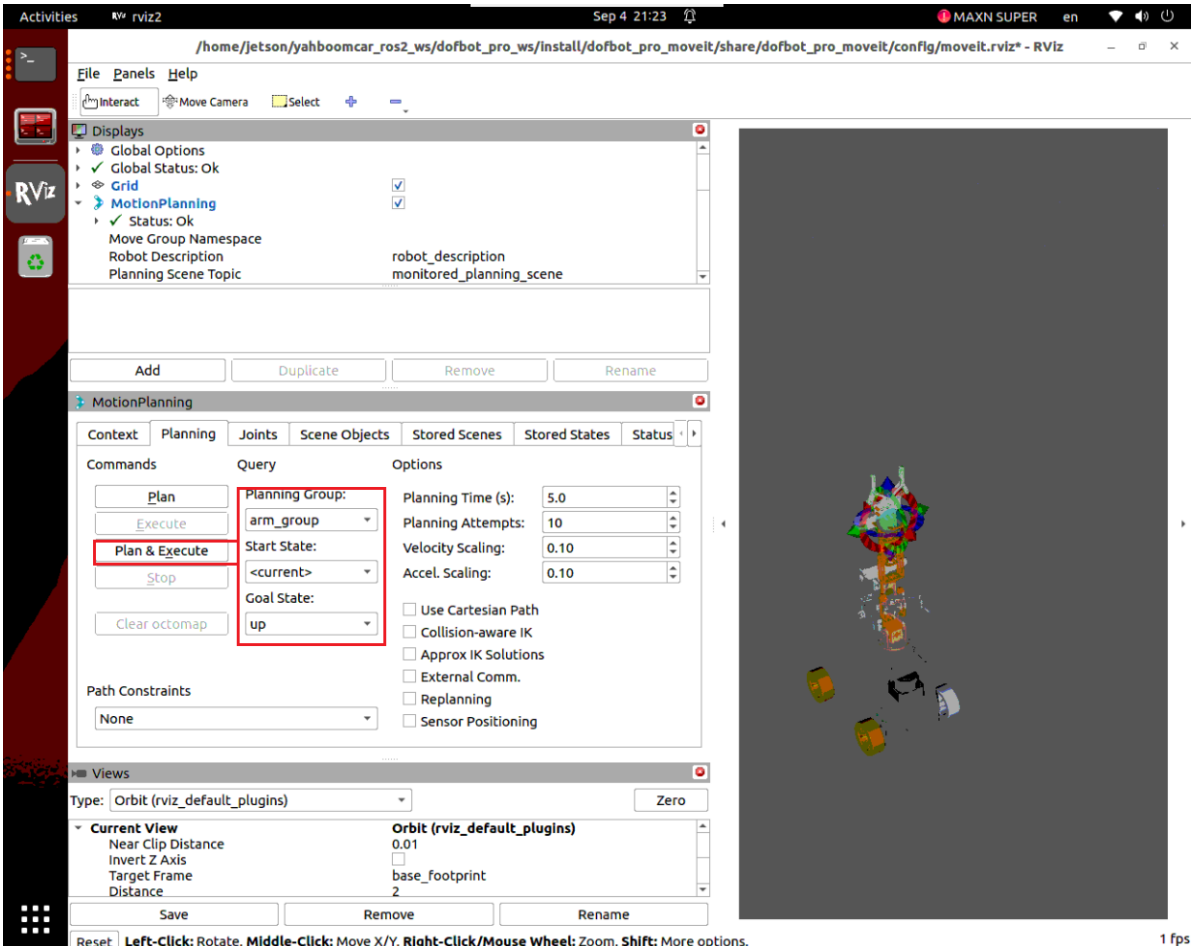
At this point, the initial pose of the simulated robot arm is init. Next, run the linkage node that drives the real machine. Be careful about the arm motion and ensure that the area around the car's robot arm is clear and safe. Open another terminal and enter the following command to launch the real-machine driving program:

```
ros2 run movelt_demo SimulationToMachine
```

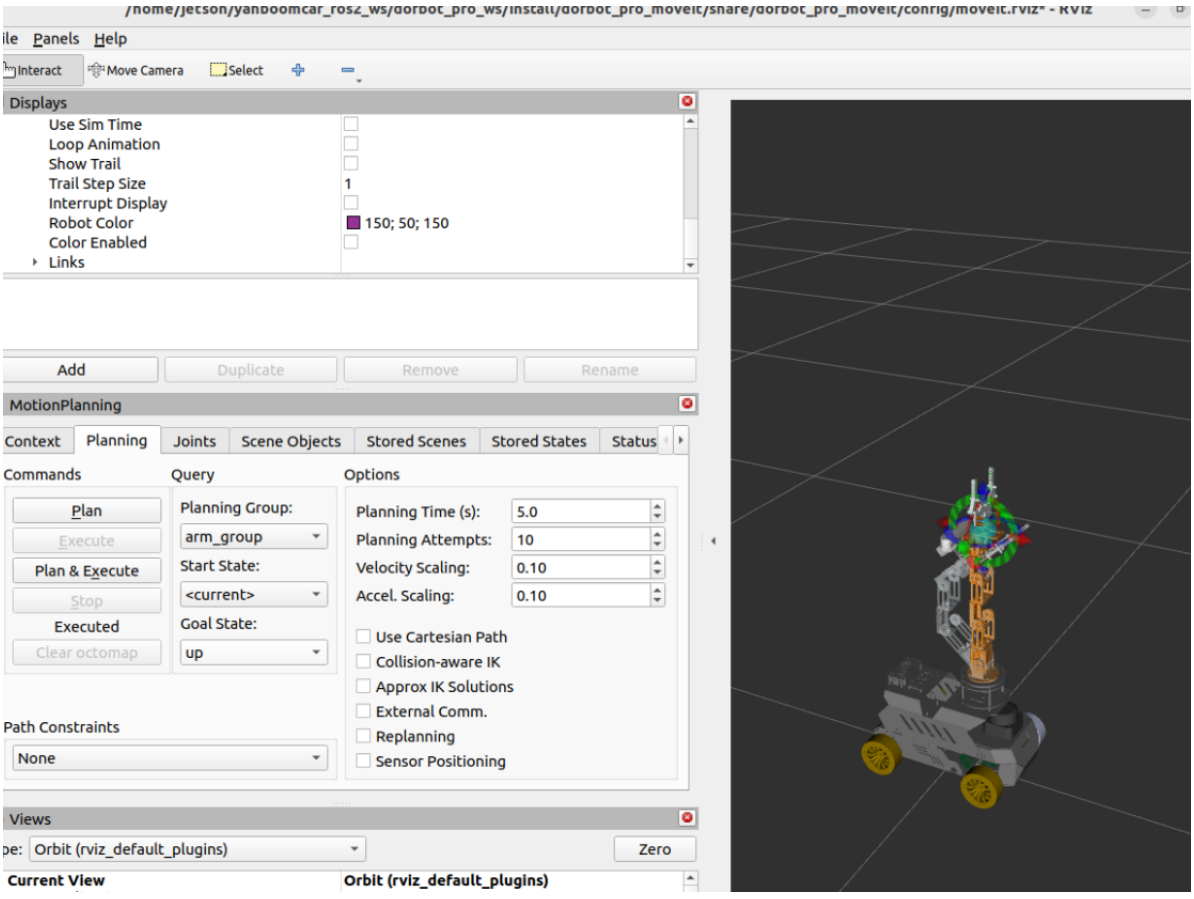
After the program runs, the real robot arm synchronizes to the initial init pose, consistent with the simulated robot arm in RViz.



At this moment, we can have the robot arm in RViz plan a motion to the preset up pose, as shown in the figure below:
Set 【Planning Group】 to `arm_group`, 【Start State】 to `<current>`, and 【Goal State】 to `up`, then click 【Plan & Execute】:



The robot arm in RViz will first perform smooth path planning and then slowly move to the up pose; meanwhile, the real robot arm on the car also synchronously moves to the same pose in real time. The final result is shown in the figure below:

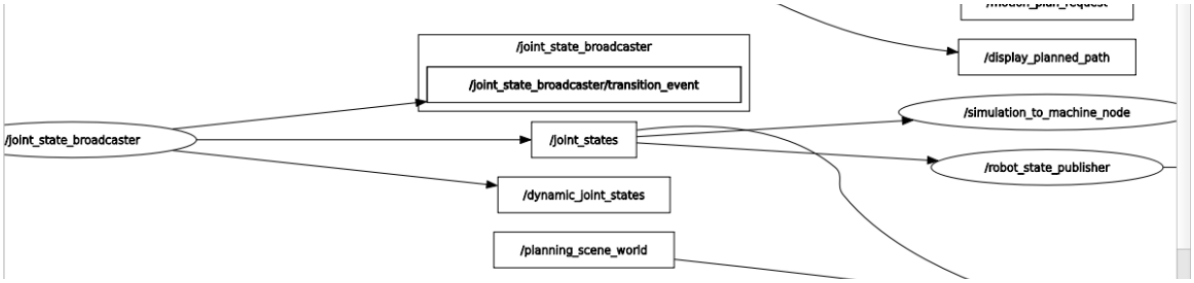


4. Node Communication

Enter the following command in the terminal to view the current node communication graph:

```
ros2 run rqt_graph rqt_graph
```

Select 【Nodes/Topics(all)】 in the upper left corner, then click the refresh button next to it to view the communication topology between the simulation node, the MoveIt node, and the real-machine driver node:



The core communication flow is: MoveIt publishes the simulated joint states to `/joint_states`, the `SimulationToMachine` node subscribes to these states, computes the angle mapping, and sends commands directly to the onboard low-level servo controller over the serial port.

5. Core Code Analysis

Program source code path: `~/yahboomcar_ros2_ws/dofbot_pro_ws/src/movelt_demo/movelt_demo/SimulationToMachine.py`

5.1 Initialization and Serial Port Controller Configuration

Import the necessary libraries, initialize the onboard low-level library `Rosmaster()` and the servo midpoint and limit parameters:

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import JointState
from Rosmaster_Lib import Rosmaster
from math import pi
import numpy as np

class SimulationToMachine(Node):
    def __init__(self):
        super().__init__('simulation_to_machine_node')

        # 1. Initialize the low-level robot arm serial port driver library
        self.Arm = Rosmaster()

        # 2. Target joint names fully aligned with the MoveIt / URDF model
        self.target_joints = [
            "arm1_Joint",   # Servo 1: base rotation
            "arm2_Joint",   # Servo 2: upper arm pitch
            "arm3_Joint",   # Servo 3: forearm pitch 1
            "arm4_Joint",   # Servo 4: forearm pitch 2
            "arm5_Joint",   # Servo 5: end-effector flip
            "grip_joint"    # Servo 6: gripper at the end
        ]
```

```
        # 3. Joint rotation direction factors and servo 90° midpoint value (0
rad in simulation corresponds to real 90°)
        self.direction = np.array([-1.0, 1.0, 1.0, 1.0, 1.0, 1.0])
        self.mid_offsets = np.array([90.0, 90.0, 90.0, 90.0, 90.0, 90.0])

        # 4. Safe limits of each servo (0° ~ 180°)
        self.servo_limits = np.array([
            [0, 180], [0, 180], [0, 180],
            [0, 180], [0, 180], [0, 180]
        ])

        # 5. Angle change dead zone threshold (0.5 degrees), to avoid sending
commands frequently due to tiny jitters
        self.angle_threshold = 0.5
        self.servo_move_time = 1000  # Servo run time (milliseconds)

        # 6. Subscribe to the simulated joint state topic published by MoveIt
        self.subscription = self.create_subscription(
            JointState,
            '/joint_states',
            self.joint_state_callback,
            10
        )

        # Return the robot arm to the initial center position
        self.Arm.set_uart_servo_angle_array(
            angle_s=[int(a) for a in self.mid_offsets],
            run_time=1000
        )
```

5.2 Joint State Callback and Servo Mapping

When MoveIt plans or drags the joints, it continuously publishes `/joint_states`. The callback function converts the simulated radians into the physical servo angles, and applies a special geometric mapping for the gripper:

```
    def joint_state_callback(self, msg: JointState):
        if not msg.name or not msg.position:
            return

        # Map joint names to positions as a dictionary
        joint_data = dict(zip(msg.name, msg.position))
        new_joint_angles = self.joint_angles[:]
        RAD2DEG = 180.0 / pi

        for i, joint_name in enumerate(self.target_joints):
            if joint_name in joint_data:
                rad_value = joint_data[joint_name]
                # Convert radians to degrees, multiply by the rotation direction
and add the 90 degree midpoint value
                deg_value = rad_value * RAD2DEG * self.direction[i]
                servo_angle = deg_value + self.mid_offsets[i]

                # Special stroke mapping for the gripper (grip_joint)
                if joint_name == "grip_joint":
                    grip_rad = joint_data[joint_name]
```

```
                    grip_deg = grip_rad * RAD2DEG + 180
                    servo_angle = np.interp(grip_deg, [90, 180], [180, 30])

                # Strictly clamp within the [0, 180] range
                servo_angle = np.clip(
                    servo_angle,
                    self.servo_limits[i][0],
                    self.servo_limits[i][1]
                )
                new_joint_angles[i] = servo_angle

        # Check whether the angle change exceeds the dead zone threshold; if the
change is significant, send serial port control
        angle_changed = any(
            abs(new - old) > self.angle_threshold
            for new, old in zip(new_joint_angles, self.last_joint_angles)
        )

        if angle_changed:
            servo_angles = [int(round(a)) for a in new_joint_angles]
            self.Arm.set_uart_servo_angle_array(
                angle_s=servo_angles,
                run_time=self.servo_move_time
            )
            self.last_joint_angles = new_joint_angles[:]

        self.joint_angles = new_joint_angles
```